

Centralized Log Analyzer & Intelligent Alerting Platform

1. Project Overview

Objective

The objective of this project is to build a **Centralized Log Analyzer & Intelligent Alerting Platform** that serves as a common logging and monitoring solution for all applications developed within the company.

Instead of implementing separate logging systems for every project, each application will integrate with a common logging SDK or API. The centralized platform will collect, process, analyze, categorize, and monitor logs in real time while automatically identifying crashes, bugs, anomalies, and performance issues. It will notify the appropriate project owners and developers whenever critical incidents occur.

This platform aims to provide a single source of truth for monitoring all company applications.

2. Current Challenges

Currently, every project uses different logging approaches such as:

- Console logging
- Winston
- Bunyan
- Log4j
- Serilog
- Firebase Crashlytics
- Sentry
- Custom logging implementations

This creates several operational challenges:

- No centralized monitoring across projects

- Different logging formats and standards
- Difficult to search logs across applications
- Manual investigation of production issues
- Duplicate errors appearing multiple times
- No automatic bug detection
- No centralized alerting
- Difficult root cause analysis
- Increased debugging time

3. Proposed Solution

Develop a centralized logging platform where every application sends logs to a common endpoint.

Example:

```
Logger.info("User Logged In")
```

```
Logger.error(error)
```

```
Logger.warning("Payment Failed")
```

or

```
POST /api/logs
```

The platform will then:

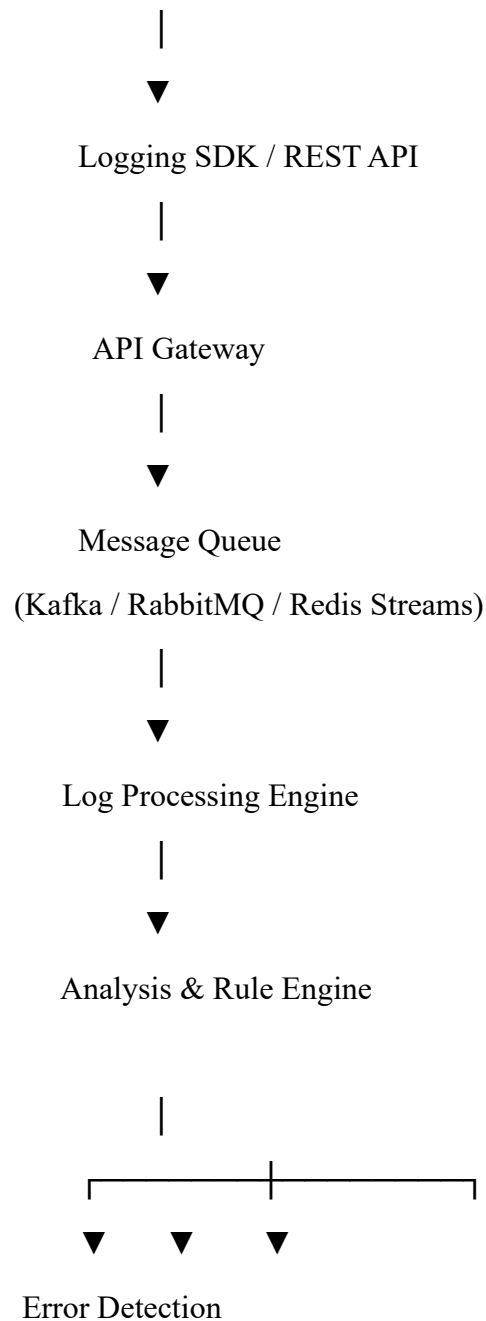
- Collect logs
- Normalize log formats
- Store logs centrally
- Analyze logs
- Detect crashes
- Detect anomalies
- Identify duplicate issues
- Generate alerts

- Notify responsible project owners
- Generate AI-assisted summaries and root cause analysis

4. High-Level Architecture

Applications

(Web, Mobile, Backend APIs, Microservices)



AI Analysis

Performance Monitoring



Central Log Database



Dashboard & Alert Engine



Email | Slack | Teams | WhatsApp | SMS

5. Core Modules

Module 1 – Logging SDK

A lightweight SDK will be developed that can be integrated into applications.

Supported platforms:

- Node.js
- React
- Flutter
- Java
- .NET
- Python

Responsibilities:

- Capture logs
- Capture exceptions
- Capture API failures

- Capture device/browser information
- Automatically send logs to the platform

Module 2 – Log Ingestion API

A centralized API responsible for receiving logs from applications.

Responsibilities:

- Authentication
- Validate incoming requests
- Normalize log data
- Forward logs to processing queue

Example payload:

```
{  
  "project": "Website",  
  "environment": "Production",  
  "service": "Authentication",  
  "level": "ERROR",  
  "message": "Invalid Token",  
  "stackTrace": "...",  
  "timestamp": "..."  
}
```

Module 3 – Message Queue

Recommended:

- Apache Kafka
- RabbitMQ
- Redis Streams

Purpose:

- Handle millions of logs efficiently
- Prevent API bottlenecks
- Enable scalable processing

Module 4 – Log Processing Engine

Responsibilities:

- Normalize log formats
- Remove duplicates
- Categorize logs
- Generate fingerprints
- Enrich logs with metadata

Example:

ERROR

ERR

Fatal

↓

Critical

Module 5 – Log Analyzer

Core responsibilities:

- Crash detection
- Exception detection
- API failure detection
- Performance analysis
- Database error detection
- Authentication failures
- Service health monitoring

- Error frequency analysis

Module 6 – AI Analysis Engine

AI can provide:

- Root cause analysis
- Duplicate issue detection
- Severity prediction
- Incident summaries
- Suggested fixes
- Trend analysis

Example:

Issue:

Payment Failure

Possible Cause:

Database timeout

Affected APIs:

Checkout

Payment

Users Impacted:

350

Severity:

Critical

Module 7 – Alert Engine

Automatically generates alerts based on configurable rules.

Examples:

- Database Down

- API Failure Rate
- Crash Spike
- Authentication Failure
- Memory Leak
- CPU Spike

Notification Channels:

- Email
- Slack
- Microsoft Teams
- WhatsApp
- SMS
- Webhooks

Module 8 – Dashboard

The dashboard provides a centralized view of all applications.

Widgets:

- Live Logs
- Error Distribution
- Active Alerts
- Top Crashes
- API Health
- Response Time
- Project Health
- Service Status
- Performance Metrics
- Recent Deployments

6. Database Design

Projects

- Project ID
- Name
- Owner
- Environment
- Status

Applications

- Application ID
- Project ID
- Version
- Platform

Logs

- Log ID
- Project
- Service
- Severity
- Message
- Stack Trace
- Device
- Browser
- Timestamp

Alerts

- Alert ID
- Severity
- Assigned To
- Status

- Created Time

Users

- User ID
- Name
- Email
- Role
- Assigned Projects

7. Major Features

Centralized Log Collection

Collect logs from every company application.

Advanced Search

Search using:

- Project
- Environment
- Date
- Severity
- API
- Service
- Device
- User
- Browser

Live Log Streaming

Real-time log monitoring.

Crash Grouping

Instead of displaying 10,000 identical errors, the system groups them into a single incident with occurrence counts.

Performance Monitoring

Monitor:

- Response Time
- Memory Usage
- CPU Usage
- Database Queries
- API Latency
- Network Performance

Release Monitoring

Compare application versions to identify regressions after deployments.

Auto Ticket Creation

Automatically create tickets in:

- Jira
- GitHub
- Azure DevOps
- Linear

User Session Replay (Future)

Replay user interactions leading to an issue for easier debugging.

8. Severity Levels

- DEBUG
- INFO
- WARNING
- ERROR

- CRITICAL
- FATAL

9. Recommended Technology Stack

Layer	Technology
Frontend	React + TypeScript
Backend	NestJS / Node.js
Database	MongoDB
Queue	Apache Kafka / RabbitMQ
Cache	Redis
AI	Python + FastAPI + OpenAI / Ollama
Authentication	JWT + API Keys
Storage	Amazon S3 / MinIO
Deployment	Docker + Kubernetes
Monitoring	Prometheus + Grafana

10. Implementation Roadmap

Phase 1 – Minimum Viable Product (4–6 Weeks)

- User Authentication
- Project Registration
- Log Ingestion API
- Logging SDK (Node.js & React)
- Central Log Storage
- Dashboard
- Search & Filtering

- Email Alerts

Phase 2 – Intelligent Monitoring (3–4 Weeks)

- Error Grouping
- Rule-Based Alert Engine
- Slack & Teams Notifications
- Performance Monitoring
- Release Tracking

Phase 3 – AI Features (4–6 Weeks)

- AI Incident Summary
- Root Cause Analysis
- Duplicate Detection
- Severity Prediction
- Weekly Health Reports

Phase 4 – Enterprise Features

- Distributed Tracing
- Session Replay
- Kubernetes Integration
- CI/CD Integration
- Git Correlation
- Predictive Analytics
- Multi-Tenant Support

11. Expected Benefits

- One centralized logging platform for all projects
- Faster debugging and troubleshooting
- Standardized logging across teams

- Real-time monitoring
- Automatic incident detection
- AI-assisted root cause analysis
- Reduced Mean Time to Resolution (MTTR)
- Improved system reliability
- Better visibility into application health
- Scalable solution for future company growth

12. Future Scope

- Machine Learning–based anomaly detection
- Predictive failure analysis
- Deployment impact analysis
- Git commit correlation
- Distributed tracing
- User session replay
- Incident Timeline Generation
- AI chatbot for log queries
- Automatic root cause reports
- Executive health dashboards

Conclusion

The proposed Centralized Log Analyzer & Intelligent Alerting Platform will provide a unified observability solution for all company applications. It will replace fragmented logging systems with a standardized, scalable platform capable of collecting logs, detecting issues, analyzing incidents using AI, and notifying the appropriate teams in real time. The platform is designed to evolve from a centralized logging solution into a complete observability system supporting logs, metrics, traces, alerts, and intelligent diagnostics.